

R Lab Week 5

Robert Gulotty

1/26/2017

```
library(purrr)
library(dplyr, warn.conflicts = F)
library(devtools)
# install_github('jennybc/repurrrsive')
library(repurrrsive)
library(broom)
library(ggplot2)
```

Lists in R

Today we learn about functional programming and purrr. R works well with “lists”. A list is a container for all kinds of mixed data types. The repurrrsive package has a list in it called `got_chars`. This is a list of characters from A Game of Thrones, a popular book, and inspiration for a violent TV series.

We can use the `str()` function to display its structure. In this case, it is a list length 29 of lists length 18.

```
length(got_chars)
```

```
## [1] 29
```

Lists are like a railroad train, each element of the list is like a car of the train, the more elements, the longer the train. If we use a single square bracket, we get the first car of the train. A single car train (a single element list) is still a train (a list).

```
got_chars[1]
```

```
## [[1]]
## [[1]]$url
## [1] "http://www.anapioficeandfire.com/api/characters/1022"
##
## [[1]]$id
## [1] 1022
##
## [[1]]$name
## [1] "Theon Greyjoy"
##
## [[1]]$gender
## [1] "Male"
##
## [[1]]$culture
## [1] "Ironborn"
##
## [[1]]$born
## [1] "In 278 AC or 279 AC, at Pyke"
##
## [[1]]$died
## [1] ""
```

```
##
## [[1]]$alive
## [1] TRUE
##
## [[1]]$titles
## [1] "Prince of Winterfell"
## [2] "Captain of Sea Bitch"
## [3] "Lord of the Iron Islands (by law of the green lands)"
##
## [[1]]$aliases
## [1] "Prince of Fools" "Theon Turncloak" "Reek" "Theon Kinslayer"
##
## [[1]]$father
## [1] ""
##
## [[1]]$mother
## [1] ""
##
## [[1]]$spouse
## [1] ""
##
## [[1]]$allegiances
## [1] "House Greyjoy of Pyke"
##
## [[1]]$books
## [1] "A Game of Thrones" "A Storm of Swords" "A Feast for Crows"
##
## [[1]]$povBooks
## [1] "A Clash of Kings" "A Dance with Dragons"
##
## [[1]]$tvSeries
## [1] "Season 1" "Season 2" "Season 3" "Season 4" "Season 5" "Season 6"
##
## [[1]]$playedBy
## [1] "Alfie Allen"
```

This gives us the entry for Theon Greyjoy.

```
str(got_chars[1])
```

```
## List of 1
## $ :List of 18
## ..$ url      : chr "http://www.anapiofire.com/api/characters/1022"
## ..$ id       : int 1022
## ..$ name     : chr "Theon Greyjoy"
## ..$ gender   : chr "Male"
## ..$ culture  : chr "Ironborn"
## ..$ born     : chr "In 278 AC or 279 AC, at Pyke"
## ..$ died     : chr ""
## ..$ alive    : logi TRUE
## ..$ titles   : chr [1:3] "Prince of Winterfell" "Captain of Sea Bitch" "Lord of the Iron Islands"
## ..$ aliases  : chr [1:4] "Prince of Fools" "Theon Turncloak" "Reek" "Theon Kinslayer"
## ..$ father   : chr ""
## ..$ mother   : chr ""
## ..$ spouse   : chr ""
```

```
## ..$ allegiances: chr "House Greyjoy of Pyke"
## ..$ books      : chr [1:3] "A Game of Thrones" "A Storm of Swords" "A Feast for Crows"
## ..$ povBooks   : chr [1:2] "A Clash of Kings" "A Dance with Dragons"
## ..$ tvSeries   : chr [1:6] "Season 1" "Season 2" "Season 3" "Season 4" ...
## ..$ playedBy   : chr "Alfie Allen"
```

Note it says “list of 1”.

If we want the content of an entry in a list, we use the double `[[ ]]`. This is like the contents of the railroad car.

```
got_chars[[1]]
```

```
## $url
## [1] "http://www.anapiofficeandfire.com/api/characters/1022"
##
## $id
## [1] 1022
##
## $name
## [1] "Theon Greyjoy"
##
## $gender
## [1] "Male"
##
## $culture
## [1] "Ironborn"
##
## $born
## [1] "In 278 AC or 279 AC, at Pyke"
##
## $died
## [1] ""
##
## $alive
## [1] TRUE
##
## $titles
## [1] "Prince of Winterfell"
## [2] "Captain of Sea Bitch"
## [3] "Lord of the Iron Islands (by law of the green lands)"
##
## $aliases
## [1] "Prince of Fools" "Theon Turncloak" "Reek"          "Theon Kinslayer"
##
## $father
## [1] ""
##
## $mother
## [1] ""
##
## $spouse
## [1] ""
##
## $allegiances
## [1] "House Greyjoy of Pyke"
```

```
##
## $books
## [1] "A Game of Thrones" "A Storm of Swords" "A Feast for Crows"
##
## $povBooks
## [1] "A Clash of Kings"      "A Dance with Dragons"
##
## $tvSeries
## [1] "Season 1" "Season 2" "Season 3" "Season 4" "Season 5" "Season 6"
##
## $playedBy
## [1] "Alfie Allen"
```

```
str(got_chars[[1]])
```

```
## List of 18
## $ url      : chr "http://www.anapiofireandfire.com/api/characters/1022"
## $ id       : int 1022
## $ name     : chr "Theon Greyjoy"
## $ gender   : chr "Male"
## $ culture  : chr "Ironborn"
## $ born     : chr "In 278 AC or 279 AC, at Pyke"
## $ died     : chr ""
## $ alive    : logi TRUE
## $ titles   : chr [1:3] "Prince of Winterfell" "Captain of Sea Bitch" "Lord of the Iron Islands (b
## $ aliases  : chr [1:4] "Prince of Fools" "Theon Turncloak" "Reek" "Theon Kinslayer"
## $ father   : chr ""
## $ mother   : chr ""
## $ spouse   : chr ""
## $ allegiances: chr "House Greyjoy of Pyke"
## $ books    : chr [1:3] "A Game of Thrones" "A Storm of Swords" "A Feast for Crows"
## $ povBooks : chr [1:2] "A Clash of Kings" "A Dance with Dragons"
## $ tvSeries : chr [1:6] "Season 1" "Season 2" "Season 3" "Season 4" ...
## $ playedBy : chr "Alfie Allen"
```

Notice in this case, we get another list, which is the first element of `got_chars`, and it has a length of 18.

Finally, we can get the interior of this list using another square bracket or double square bracket.

```
got_chars[[1]][1]
```

```
## $url
## [1] "http://www.anapiofireandfire.com/api/characters/1022"
```

```
got_chars[[1]][[1]]
```

```
## [1] "http://www.anapiofireandfire.com/api/characters/1022"
```

A shortcut for the second method, when we know the names of the elements is to use the `$`. This should look familiar. This is because `data.frames` are lists!

```
got_chars[[1]]$url
```

```
## [1] "http://www.anapiofireandfire.com/api/characters/1022"
```

Purrr

Ok, so we have a list of lists of vectors. To manipulate them more easily, we will learn about the map functions in purrr. Suppose we want to just return a named part of a list. We can do so with the `map()` function.

```
map(got_chars[1:3], "gender")
```

```
## [[1]]
## [1] "Male"
##
## [[2]]
## [1] "Male"
##
## [[3]]
## [1] "Male"
```

The basic map function returns a list. `map_x` returns an object of type x.

```
map_chr(got_chars, "gender")
```

```
## [1] "Male" "Male" "Male" "Male" "Male" "Male" "Male"
## [8] "Female" "Female" "Male" "Female" "Male" "Female" "Male"
## [15] "Male" "Male" "Female" "Female" "Female" "Male" "Male"
## [22] "Male" "Male" "Male" "Male" "Female" "Male" "Male"
## [29] "Female"
```

```
map_chr(got_chars, "name")
```

```
## [1] "Theon Greyjoy" "Tyrion Lannister" "Victarion Greyjoy"
## [4] "Will" "Areo Hotah" "Chett"
## [7] "Cressen" "Arianne Martell" "Daenerys Targaryen"
## [10] "Davos Seaworth" "Arya Stark" "Arys Oakheart"
## [13] "Asha Greyjoy" "Barristan Selmy" "Varamyr"
## [16] "Brandon Stark" "Brienne of Tarth" "Catelyn Stark"
## [19] "Cersei Lannister" "Eddard Stark" "Jaime Lannister"
## [22] "Jon Connington" "Jon Snow" "Aeron Greyjoy"
## [25] "Kevan Lannister" "Melisandre" "Merrett Frey"
## [28] "Quentyn Martell" "Sansa Stark"
```

Ok, so why would we use such a thing? We can, for example, run a bunch of models at once.

Consider the following dataset:

```
head(mtcars)
```

```
##           mpg  cyl  disp  hp  drat    wt  qsec vs  am  gear  carb
## Mazda RX4      21.0   6  160 110 3.90 2.620 16.46 0   1    4    4
## Mazda RX4 Wag  21.0   6  160 110 3.90 2.875 17.02 0   1    4    4
## Datsun 710      22.8   4  108  93 3.85 2.320 18.61 1   1    4    1
## Hornet 4 Drive  21.4   6  258 110 3.08 3.215 19.44 1   0    3    1
## Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02 0   0    3    2
## Valiant         18.1   6  225 105 2.76 3.460 20.22 1   0    3    1
```

Each observation is a type of car, and we have information about fuel efficiency, mpg, the size of the engine, both in count and in terms of volume and weight, and some information about horsepower.

One of the main determinants of fuel efficiency is the number of cylinders, so let's say we want to do an analysis stratified or grouped by cyl. The `split` command takes a column, here identified using the `$` shortcut, and creates a list.

```
mtcars %>% split(.$cyl)
```

```
## $`4`
##           mpg cyl  disp  hp drat   wt  qsec vs am gear carb
## Datsun 710   22.8  4 108.0  93 3.85 2.320 18.61  1  1    4    1
## Merc 240D   24.4  4 146.7  62 3.69 3.190 20.00  1  0    4    2
## Merc 230    22.8  4 140.8  95 3.92 3.150 22.90  1  0    4    2
## Fiat 128    32.4  4  78.7  66 4.08 2.200 19.47  1  1    4    1
## Honda Civic 30.4  4  75.7  52 4.93 1.615 18.52  1  1    4    2
## Toyota Corolla 33.9  4  71.1  65 4.22 1.835 19.90  1  1    4    1
## Toyota Corona 21.5  4 120.1  97 3.70 2.465 20.01  1  0    3    1
## Fiat X1-9   27.3  4  79.0  66 4.08 1.935 18.90  1  1    4    1
## Porsche 914-2 26.0  4 120.3  91 4.43 2.140 16.70  0  1    5    2
## Lotus Europa 30.4  4  95.1 113 3.77 1.513 16.90  1  1    5    2
## Volvo 142E  21.4  4 121.0 109 4.11 2.780 18.60  1  1    4    2
##
## $`6`
##           mpg cyl  disp  hp drat   wt  qsec vs am gear carb
## Mazda RX4   21.0  6 160.0 110 3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag 21.0  6 160.0 110 3.90 2.875 17.02  0  1    4    4
## Hornet 4 Drive 21.4  6 258.0 110 3.08 3.215 19.44  1  0    3    1
## Valiant     18.1  6 225.0 105 2.76 3.460 20.22  1  0    3    1
## Merc 280     19.2  6 167.6 123 3.92 3.440 18.30  1  0    4    4
## Merc 280C    17.8  6 167.6 123 3.92 3.440 18.90  1  0    4    4
## Ferrari Dino  19.7  6 145.0 175 3.62 2.770 15.50  0  1    5    6
##
## $`8`
##           mpg cyl  disp  hp drat   wt  qsec vs am gear carb
## Hornet Sportabout 18.7  8 360.0 175 3.15 3.440 17.02  0  0    3    2
## Duster 360       14.3  8 360.0 245 3.21 3.570 15.84  0  0    3    4
## Merc 450SE       16.4  8 275.8 180 3.07 4.070 17.40  0  0    3    3
## Merc 450SL       17.3  8 275.8 180 3.07 3.730 17.60  0  0    3    3
## Merc 450SLC      15.2  8 275.8 180 3.07 3.780 18.00  0  0    3    3
## Cadillac Fleetwood 10.4  8 472.0 205 2.93 5.250 17.98  0  0    3    4
## Lincoln Continental 10.4  8 460.0 215 3.00 5.424 17.82  0  0    3    4
## Chrysler Imperial 14.7  8 440.0 230 3.23 5.345 17.42  0  0    3    4
## Dodge Challenger  15.5  8 318.0 150 2.76 3.520 16.87  0  0    3    2
## AMC Javelin      15.2  8 304.0 150 3.15 3.435 17.30  0  0    3    2
## Camaro Z28       13.3  8 350.0 245 3.73 3.840 15.41  0  0    3    4
## Pontiac Firebird  19.2  8 400.0 175 3.08 3.845 17.05  0  0    3    2
## Ford Pantera L   15.8  8 351.0 264 4.22 3.170 14.50  0  1    5    4
## Maserati Bora     15.0  8 301.0 335 3.54 3.570 14.60  0  1    5    8
```

```
splitmtcars <- mtcars %>% split(.$cyl)
```

Notice the `.$` construction. This is to indicate to functions that are not part of the dplyr universe, like `mutate` and `ggplot`, which always put the data command first. Here we need to specify that you want to refer to the dataset being piped into the command using `%>%`.

Now we have a list of data.frames. We can use this `map` command to fit a function to each dataframe. This `map` command returns a list as a default, but it will return a list of regression models.

```
splitmtcars %>% map(~lm(mpg ~ wt, data = .x))
```

```
## $`4`
##
## Call:
## lm(formula = mpg ~ wt, data = .x)
##
## Coefficients:
## (Intercept)          wt
##      39.571      -5.647
##
## $`6`
##
## Call:
## lm(formula = mpg ~ wt, data = .x)
##
## Coefficients:
## (Intercept)          wt
##      28.41      -2.78
##
## $`8`
##
## Call:
## lm(formula = mpg ~ wt, data = .x)
##
## Coefficients:
## (Intercept)          wt
##      23.868      -2.192
```

We can further apply map functions to the output, such as the summary command:

```
splitmtcars %>% map(~lm(mpg ~ wt, data = .x)) %>% map(summary)
```

```
## $`4`
##
## Call:
## lm(formula = mpg ~ wt, data = .x)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -4.1513 -1.9795 -0.6272  1.9299  5.2523
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   39.571      4.347    9.104 7.77e-06 ***
## wt           -5.647      1.850   -3.052  0.0137 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.332 on 9 degrees of freedom
## Multiple R-squared:  0.5086, Adjusted R-squared:  0.454
## F-statistic: 9.316 on 1 and 9 DF, p-value: 0.01374
##
```

```
##
## $`6`
##
## Call:
## lm(formula = mpg ~ wt, data = .x)
##
## Residuals:
##      Mazda RX4      Mazda RX4 Wag  Hornet 4 Drive      Valiant      Merc 280
##      -0.1250       0.5840       1.9292      -0.6897       0.3547
##      Merc 280C      Ferrari Dino
##      -1.0453      -1.0080
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   28.409      4.184   6.789 0.00105 **
## wt           -2.780      1.335  -2.083 0.09176 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.165 on 5 degrees of freedom
## Multiple R-squared:  0.4645, Adjusted R-squared:  0.3574
## F-statistic: 4.337 on 1 and 5 DF, p-value: 0.09176
##
##
## $`8`
##
## Call:
## lm(formula = mpg ~ wt, data = .x)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.1491 -1.4664 -0.8458  1.5711  3.7619
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   23.8680      3.0055   7.942 4.05e-06 ***
## wt           -2.1924      0.7392  -2.966  0.0118 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.024 on 12 degrees of freedom
## Multiple R-squared:  0.423, Adjusted R-squared:  0.3749
## F-statistic: 8.796 on 1 and 12 DF, p-value: 0.01179
```

Finally, we can pull out a element of those summary commands with a final map command. Here we are getting the r-squared, which is a measure of the total share of variation in the outcome variable, here `mpg`, that is associated with our independent variables, here just `wt`.

```
splitmtcars %>% map(~lm(mpg ~ wt, data = .x)) %>% map(summary) %>%
  map_dbl("r.squared")
```

```
##           4           6           8
## 0.5086326 0.4645102 0.4229655
```

Cool huh?

Ok, so now lets use this technique.

Re-analysis of afghanistan

```
afg <- read.csv("~/Google Drive/ChicagoSyllabi/SSI131/Rcode/afg2.csv",  
  as.is = T)
```

Here we output to a data frame using the tidy function, and then we restrict attention to the ELEVATION coefficients. Be sure to check each of the steps so you know what it is doing.

```
afg %>% split(.$LANGUAGE) %>% map(~lm(POPULATION ~ ELEVATION,  
  data = .x)) %>% map_df(tidy) %>% dplyr::filter(term == "ELEVATION")
```

##	term	estimate	std.error	statistic	p.value
## 1	ELEVATION	0.9959049	0.16868803	5.903827	6.195509e-09
## 2	ELEVATION	-0.3355707	0.15860867	-2.115715	3.438091e-02
## 3	ELEVATION	-1.4630264	0.59346860	-2.465213	1.402344e-02
## 4	ELEVATION	-0.2886591	0.15285599	-1.888438	6.000074e-02
## 5	ELEVATION	-0.3576687	0.14290268	-2.502883	1.254508e-02
## 6	ELEVATION	-0.1588966	0.12373570	-1.284161	1.996838e-01
## 7	ELEVATION	-0.2333019	0.03301250	-7.067076	1.637026e-12
## 8	ELEVATION	-0.3843047	0.26174905	-1.468218	1.429062e-01
## 9	ELEVATION	-0.2400234	0.09616942	-2.495840	1.263912e-02

Functional Programming

R supports functional programming. In practice, this means there are often opportunities to vastly improve the speed and efficiency of your code by using a custom function. Before getting there, let's review the `apply` family, which help take functions and applies it a bunch of times to a set of data. `lapply` takes a list, and applies the function to each element of the list.

```
x <- list(a = rnorm(rbinom(1, 10, 0.4)), b = rnorm(rbinom(1,
  10, 0.6)), c = rnorm(rbinom(1, 10, 0.9)))
unlist(lapply(x, mean))
```

```
##           a           b           c
## -0.6279227  0.5990578  0.2742273
```

The second most popular `apply` function is the `tapply` function, which allows you to use a function on subsets of the data, usually by group.

```
df <- data.frame(tarifftrate = c(rbeta(5, 2, 1), rbeta(2, 1, 8),
  rbeta(8, 1, 2)), country = c(rep("USA", 5), rep("UK", 2),
  rep("Japan", 8)))
tapply(df$tarifftrate, df$country, mean)
```

```
##      Japan      UK      USA
## 0.2911600 0.1447359 0.6540977
```

The `apply` family, including `tapply` and `lapply` are useful and relatively fast. They work particularly well in combination with custom functions. Custom functions allow you to combine and mix a variety of functions. One striking thing to do is to use a function which makes other functions. Here is a simple example. Suppose you want to calculate several features of some fake data

```
df2 <- data.frame(replicate(6, sample(c(1:10, NA), 6, rep = TRUE)))
names(df2) <- letters[1:6]

mean(df2$a, na.rm = TRUE)
```

```
## [1] 6.2
```

```
median(df2$a, na.rm = TRUE)
```

```
## [1] 8
```

```
sd(df2$a, na.rm = TRUE)
```

```
## [1] 3.898718
```

```
mad(df2$a, na.rm = TRUE)
```

```
## [1] 2.9652
```

```
IQR(df2$a, na.rm = TRUE)
```

```
## [1] 7
```

Notice the repeated text. We can write a function which creates functions and then applies them to the data.

```
summaries <- function(x) {  
  sumfuncs <- c(mean, median, sd, mad, IQR)  
  listout <- lapply(sumfuncs, function(f) {  
    f(x, na.rm = T)  
  })  
  names(listout) <- c("mean", "median", "sd", "mad", "IQR")  
  return(unlist(listout))  
}  
summaries(df2$a)
```

```
##      mean  median      sd      mad      IQR  
## 6.200000 8.000000 3.898718 2.965200 7.000000
```

For most purposes (when you have a standard data frame) we can use the `dplyr` summarise function, which does much the same thing as `apply`.

```
summarise(df2, mean = mean(a, na.rm = T), median = median(a,  
  na.rm = T), sd = sd(a, na.rm = T), mad = mad(a, na.rm = T),  
  IQR = IQR(a, na.rm = T))
```

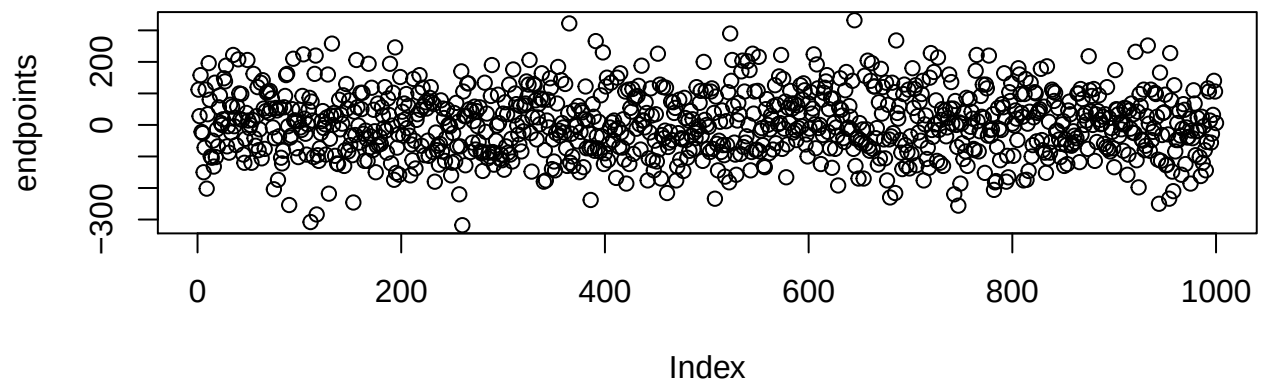
```
##      mean median      sd      mad IQR  
## 1  6.2      8 3.898718 2.9652    7
```

Again, if you groupby first, you can calculate these variables for subgroups.

Application: studying the normal distribution

Recall the small error fuction from the second R lab? If not, go back and read about it. One of the problems is that it was very slow. We can use the `replicate` command from the `apply` family to make it **vectorized** and work much faster.

```
smallerrors <- function(lengthofchain) {  
  directionoferror <- c(1, -1)  
  x <- sum(sample(directionoferror, size = lengthofchain, replace = T))  
  return(x)  
}  
  
endpoints <- replicate(1000, smallerrors(10000))  
  
plot(endpoints)
```



This was much faster, as it used vectorization, which means that it uses an `apply` function rather than a loop. The main idea is that R works better when it is not looping. In general there may be a tradeoff between the ease of writing a command and the speed that the computer can calculate the answer, but in this case there are also readability advantages to using the vectorized code.

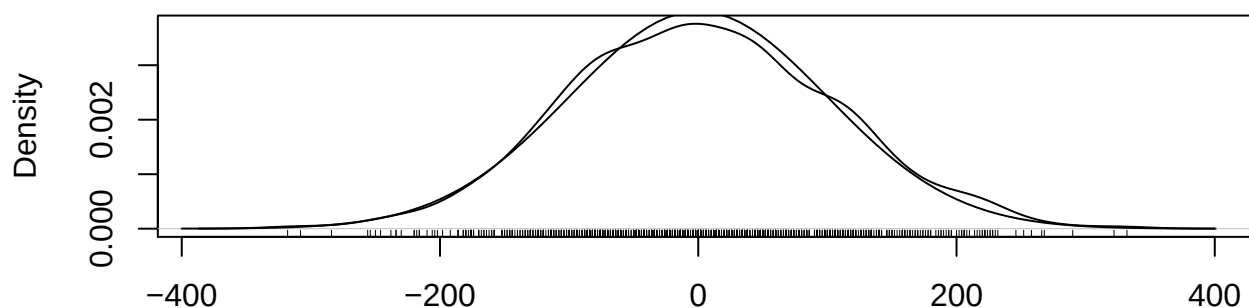
Now recall that these end points follow a normal distribution. The pdf of the normal distribution is described by the following equation:

$$f(x|\mu, \sigma^2) = \frac{1}{\sqrt{2\sigma\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Where σ is the standard deviation and μ is the mean. In this case, the mean is 0, and the standard deviation is the square root of the length of the chain, which here is 100.

```
plot(density(endpoints))
curve(dnorm(x, mean = 0, sd = 100), from = -400, to = 400, add = T)
rug(endpoints)
```

density.default(x = endpoints)



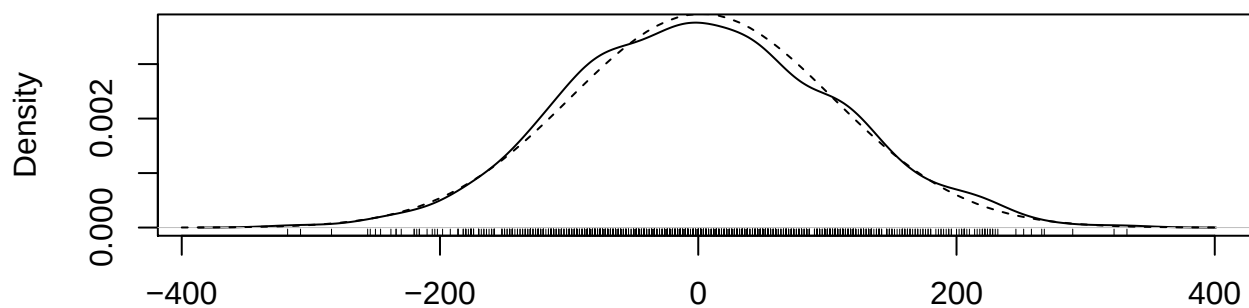
N = 1000 Bandwidth = 22.99

Notice I put a rug plot at the bottom, which shows where the points lie on the line.

Now suppose that instead of plotting the theoretical distribution, we wanted to plot the observed distribution using the normal approximation. Here we replace the theoretically known mean and standard deviation with the sampled data.

```
plot(density(endpoints))
curve(dnorm(x, mean = mean(endpoints), sd = sd(endpoints)), from = -400,
      to = 400, add = T, lty = 2)
rug(endpoints)
```

density.default(x = endpoints)



N = 1000 Bandwidth = 22.99

Try playing around with the original simulated data, and see how the observed fit of the normal does.

ADVANCED: Multivariate Distributions and mapply

Now we have seen how a distribution can be fit to a sample, but what if the data is not just a point on a line? What if, for example, we observe several characteristics at once, such as the correlation between several variables, an event in time and space, or an otherwise more complicated event. We can say that these examples make up a vector of random variables. This can be any where from a very high dimensional vector of all sorts of independent variable estimates from a regression, or the simplest case, two parameters, such as a point on a plane. If we wish to study such a system we can characterize a joint distribution using a multivariate function. One of the most useful of these is the multivariate normal distribution, which is an extension of the normal distribution. If you have k dimensional random vector $\mathbf{x} = [X_1, X_2 \dots, X_k]$ then we can say $\mathbf{x} \sim N(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, where $\boldsymbol{\Sigma}$ is a variance covariance **matrix**, and $\boldsymbol{\mu}$ is a vector of means.

The pdf of such a random vector is the following:

$$f_X(x_1, \dots, x_k) = \frac{1}{\sqrt{(2\pi)^k |\boldsymbol{\Sigma}|}} e^{-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})}$$

Where $|\boldsymbol{\Sigma}|$ is the determinant of the variance covariance matrix $\boldsymbol{\Sigma}$, and $\mathbf{x}$ and $\boldsymbol{\mu}$ are vectors. The simplest case of this is the bivariate case, where

$$\boldsymbol{\mu} = \begin{bmatrix} \mu_{X_1} \\ \mu_{X_2} \end{bmatrix}, \quad \boldsymbol{\Sigma} = \begin{pmatrix} \sigma_{X_1}^2 & \rho \sigma_{X_1} \sigma_{X_2} \\ \rho \sigma_{X_2} \sigma_{X_1} & \sigma_{X_2}^2 \end{pmatrix}$$

$$f_X(x_1, x_2) = \frac{1}{2\pi \sigma_x \sigma_y \sqrt{1 - \rho^2}} \exp \left(-\frac{1}{2(1 - \rho^2)} \left[\frac{(x - \mu_X)^2}{\sigma_X^2} + \frac{(y - \mu_Y)^2}{\sigma_Y^2} - \frac{2\rho(x - \mu_X)(y - \mu_Y)}{\sigma_X \sigma_Y} \right] \right)$$

I leave it to you to try to invert $\boldsymbol{\Sigma}$ and figure this out. In any case, we can use the built in R function to draw a sample from such a distribution. `mvdraws` is a 1000 by 2 matrix.

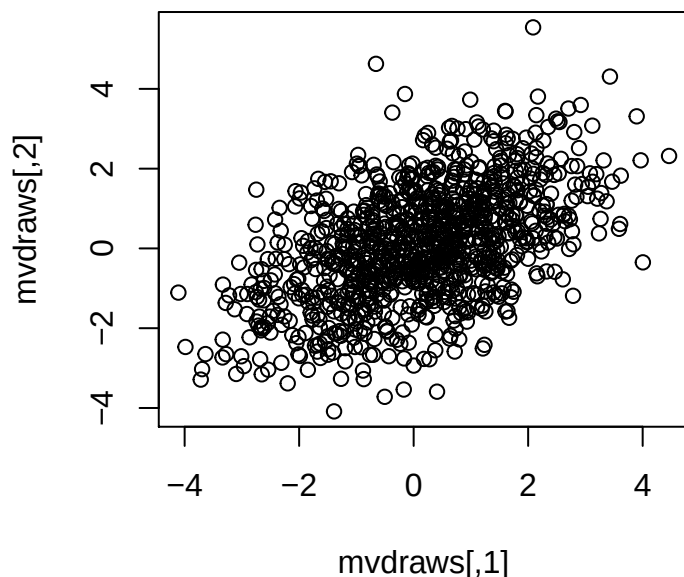
```
library(mvtnorm)

mvdraws <- rmvnorm(n = 1000, mean = c(0, 0), sigma = matrix(c(2,
  1, 1, 2), nrow = 2, byrow = T))
dim(mvdraws)

## [1] 1000    2
```

Because we are still in $\mathbf{R}^2$, we can plot these points on a plane.

```
plot(mvdraws)
```



We can work with this sort of data using the `mapply` function, which operates on multiple lists. First we calculate the sample means and the sample variance covariance matrix, and then we fit these in the multivariate normal distribution just as we did in the one dimensional case. To do so, we use the `outer` command to create all the possible combinations of `x` and `y`. Notice we are using `%%` for matrix multiplication, and `solve`, which finds the inverse of a matrix. We then plot these as ellipse using a contour plot and as a height in a perspective plot

```
samplecenter <- apply(mvdraws, 2, mean)
samplesigma <- cov(mvdraws)

sigma.inv = solve(samplesigma)

ellipse <- function(s, t) {
  u <- c(s, t) - samplecenter

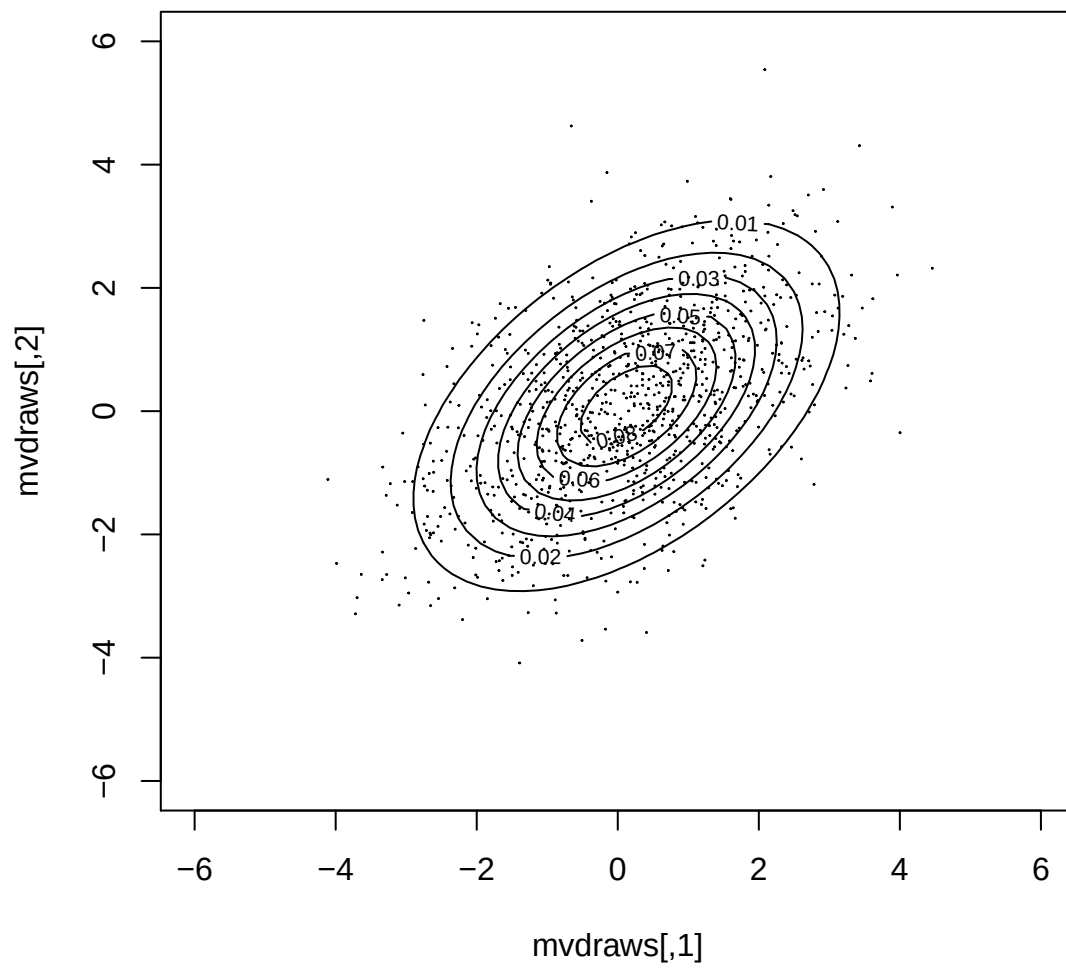
  exp(-u %*% sigma.inv %*% u/2)/(sqrt(det(samplesigma)) * (2 *
    pi)^2))
}

# Grid
n = 50
x = seq(-5, 5, length = 50)
y = seq(-5, 5, length = 50)

z = mapply(ellipse, rep(x, n), as.vector(outer(rep(0, n), y,
  `+`)))

plot(mvdraws, pch = 20, xlim = c(-6, 6), ylim = c(-6, 6), cex = 0.1)

contour(x, y, matrix(z, n, n), nlevels = 8, col = "black", add = TRUE,
  labcex = 0.7)
```



```
persp(x, y, matrix(z, n, n))
```